

Sensor-Based IoT Simulation

Light-Controlled LED Using an LDR Sensor (Arduino Uno, Proteus ISIS)

Note on screenshots: Proteus runs as a desktop application on your computer, so I cannot operate it directly or generate live screenshots myself. This document gives you the complete circuit design, the compiled-code workflow Proteus requires, and exact step-by-step instructions so you can build and run the simulation yourself in Proteus and capture your own screenshots for submission.

1. Objective

This simulation demonstrates a basic IoT-style sensing and actuation loop in Proteus ISIS: an LDR (Light Dependent Resistor) continuously senses ambient light intensity, and an Arduino Uno microcontroller processes that reading to automatically switch an LED ON in dark conditions and OFF in bright conditions. This mirrors real IoT applications such as automatic street lighting and smart indoor lighting, where sensing replaces manual switching.

2. Key Difference: How Proteus Runs Arduino Code

Unlike Tinkercad, Proteus ISIS does not have a built-in Arduino code editor. The Arduino sketch must first be written and compiled in the Arduino IDE (or Arduino Web Editor), which produces a compiled program file with a .hex extension. This .hex file is then loaded onto the virtual Arduino Uno inside Proteus. The overall workflow is therefore:

- Write the sketch in the Arduino IDE.
- Compile it (Sketch → Verify/Compile) to generate the .hex file.
- Build the circuit in Proteus ISIS.
- Load the .hex file onto the Arduino Uno part's properties in Proteus.
- Run the simulation.

3. Components Used (Proteus Part Names)

Component	Proteus Library Part Name	Purpose
Arduino Uno	ARDUINO UNO R3	Microcontroller board that reads the sensor and drives the LED
LDR	LDR (in Optoelectronics / search "LDR")	Sensor whose resistance changes with light intensity
Resistor	RES, 10k	Forms voltage divider with the LDR for analog reading
Resistor	RES, 330R	Limits current through the LED
LED	LED-RED	Output / actuator that turns ON or OFF

Power source	Provided by Arduino's 5V/GND pins	Supplies the divider circuit
--------------	-----------------------------------	------------------------------

Note: Some Proteus installations do not include a native LDR part by default. If LDR is not found in the Pick Devices search, you can substitute a potentiometer (POT-HG) wired as a variable voltage divider to manually simulate changing light levels, using the same circuit logic and code.

4. Circuit Diagram

The LDR and the 10 k Ω resistor form a voltage divider connected between 5V and GND, with the midpoint connected to analog pin A0 of the Arduino. The LED is connected through a 330 Ω resistor between pin 13 and GND. The same wiring logic applies directly in Proteus.

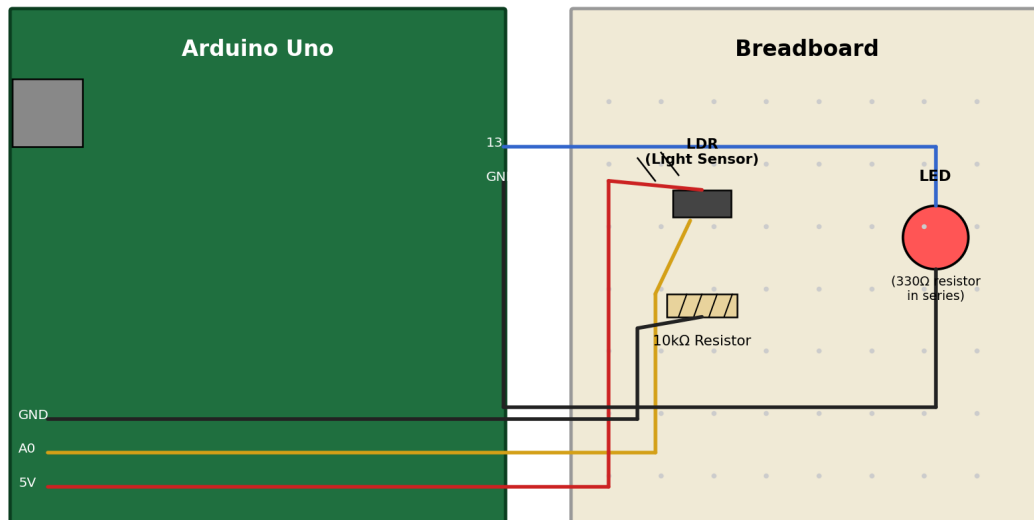


Figure 1: Circuit wiring diagram — LDR voltage divider feeding A0; LED driven from pin 13.

Wiring Connections Summary

- Arduino 5V → one leg of LDR
- Other leg of LDR → Arduino A0 AND → top of 10 k Ω resistor
- Bottom of 10 k Ω resistor → Arduino GND
- Arduino pin 13 → 330 Ω resistor → LED anode (long leg / positive)
- LED cathode (short leg / negative) → Arduino GND

5. Arduino Code

```
int ldrPin = A0;      // LDR connected to analog pin A0
int ledPin = 13;      // LED connected to digital pin 13
int threshold = 500;  // Light level threshold (0-1023)
int lightValue = 0;   // Variable to store sensor reading

void setup() {
  pinMode(ledPin, OUTPUT); // Set LED pin as output
  Serial.begin(9600);      // Start serial communication for monitoring
}

void loop() {
  lightValue = analogRead(ldrPin); // Read LDR value (0-1023)
  Serial.println(lightValue);      // Print value to Serial Monitor

  if (lightValue < threshold) {
    digitalWrite(ledPin, HIGH); // Dark detected -> turn LED ON
  } else {
    digitalWrite(ledPin, LOW);  // Bright detected -> turn LED OFF
  }

  delay(200); // Small delay for stable readings
}
```

6. Generating the .hex File for Proteus

- 1 Open the Arduino IDE on your computer (download free from [arduino.cc](https://www.arduino.cc) if not installed).
- 2 Paste the code from Section 5 into a new sketch.
- 3 Go to File → Preferences, and check "Show verbose output during compilation" — this makes the .hex file path visible after compiling.
- 4 Select Tools → Board → Arduino Uno (no need to select a Port or upload to real hardware).
- 5 Click the Verify/Compile button (checkmark icon), or Sketch → Verify/Compile.
- 6 After compilation finishes, scroll the black output console at the bottom — it will show a path ending in a file such as sketch_name.ino.hex inside a temporary build folder (e.g., C:\Users\YourName\AppData\Local\Temp\arduino_build_XXXXXX\).
- 7 Copy that .hex file to an easy-to-find folder (e.g., your Desktop) — you will point Proteus to this file in the next section.

7. Code Explanation (Line by Line)

Variable declarations: ldrPin and ledPin store the pin numbers used for the sensor and the LED. threshold defines the light level below which the surroundings are considered "dark". lightValue holds the current sensor reading.

setup() function: Runs once when the Arduino starts. pinMode(ledPin, OUTPUT) configures pin 13 to send voltage out to the LED. Serial.begin(9600) opens a communication channel so sensor

readings can be viewed live on Proteus's Virtual Terminal.

analogRead(ldrPin): Reads the voltage at A0, which varies depending on how much light is hitting the LDR. The LDR's resistance drops in bright light and rises in darkness, changing the voltage at the midpoint of the divider. The Arduino converts this voltage into a number between 0 (no voltage) and 1023 (full voltage).

Serial.println(lightValue): Prints the current reading over the serial port so it can be observed on a Virtual Terminal connected to the Arduino's TX pin in Proteus.

if (lightValue < threshold): Compares the live reading to the threshold value. A lower reading means less light is reaching the LDR (a darker environment), since less light generally results in higher LDR resistance and therefore a lower voltage at A0 in this wiring configuration.

digitalWrite(ledPin, HIGH/LOW): Sends 5V (HIGH) to turn the LED on when it is dark, or 0V (LOW) to switch it off when it is bright, directly actuating the output based on the sensor input.

delay(200): Pauses briefly between readings to prevent the LED from flickering due to rapid, noisy fluctuations in the sensor data.

8. Step-by-Step: Building This in Proteus ISIS

- 1 Open Proteus, click ISIS Schematic Capture to create a new project.
- 2 Click the "Pick Devices" button (P) on the left toolbar. Search for and add: ARDUINO UNO R3, LDR, RES (x2), and LED-RED to your parts list.
- 3 Drag the Arduino Uno, the LDR, both resistors, and the LED onto the schematic sheet.
- 4 Set one resistor's value to 10k and the other to 330R by double-clicking each part and editing the Resistance field in the properties dialog.
- 5 Wire the components exactly as shown in Figure 1 and the Wiring Connections Summary in Section 4: 5V to LDR, LDR junction to A0 and to the 10k resistor, 10k resistor to GND, pin 13 through 330R to the LED anode, and LED cathode to GND.
- 6 Add a GROUND terminal (from the Terminals mode icon) and connect it to the Arduino's GND pin and the circuit's GND rail so all ground connections are common.
- 7 Double-click the Arduino Uno part on the schematic. In the "Program File" field of the properties dialog that opens, click the folder icon and browse to the .hex file you generated in Section 6. Click OK. (TAKE SCREENSHOT 1 here: this properties dialog showing the loaded hex path.)
- 8 (Optional) Add a VIRTUAL TERMINAL part, connect its RXD to the Arduino's TX (pin 1/D1), and set its baud rate to 9600 to view live LDR readings during simulation.
- 9 Click the "Play" (Run Simulation) button at the bottom-left of the Proteus window. (TAKE SCREENSHOT 2 here: full schematic running, LED state visible, Virtual Terminal open if added.)
- 10 While the simulation is running, double-click the LDR component and adjust its resistance/light property (or drag its slider, depending on Proteus version) to simulate darkness — the LED should turn ON. (TAKE SCREENSHOT 3 here: LED ON in dark condition.)
- 11 Adjust the LDR back toward a bright-light value — the LED should turn OFF. (TAKE SCREENSHOT 4 here: LED OFF in bright condition.)
- 12 Click the Stop button to end the simulation, then save your project (File → Save Design) for your records.

9. Working Principle (IoT Concept)

This circuit follows the classic IoT sense-process-act loop. The LDR (sensing layer) continuously gathers environmental data. The Arduino (processing layer) reads this data and applies a decision rule. The LED (actuation layer) responds automatically without human intervention. In a real IoT deployment, this same logic could be extended by sending the light level to a cloud dashboard over Wi-Fi (e.g., using an ESP8266/ESP32 instead of the Uno), enabling remote monitoring and control of the lighting system from a phone or web app.

10. Expected Results

LDR Condition	Analog Reading (approx.)	LED State
Bright light	Greater than 500	OFF
Dark / covered	Less than 500	ON

Note: The exact threshold value (500) may need adjustment depending on the LDR model's default resistance curve in your Proteus library; tune it by watching live readings on the Virtual Terminal and adjusting the threshold variable in the code, then recompiling and reloading the .hex file.

11. Common Proteus Troubleshooting

- **"No COM port" or programming errors:** Not needed for simulation — you only load the .hex file via the Arduino's Program File property, not by uploading over USB.
- **LED never lights up:** Double-check the LED is oriented correctly (anode toward the resistor/pin 13 side) and that GND connections are common across the whole circuit.
- **LDR part missing:** Ensure your Proteus library includes optoelectronic parts, or substitute a potentiometer as described in Section 3.
- **Simulation runs but values don't change:** Confirm you recompiled and reloaded the .hex file after any code edits — Proteus does not auto-detect code changes.

12. Conclusion

This simulation successfully demonstrates a basic light-sensing IoT system using an LDR and an Arduino Uno, built and tested in Proteus ISIS. It illustrates the foundational sense-process-act pipeline used across far more complex IoT applications, from smart street lighting to greenhouse climate control, and shows how a low-cost sensor and a simple conditional rule can automate a real-world task without any manual switching.